

✓ Lab 5: Sequence Labeling

605.646 Natural Language Processing

Fall 2025

This is a two-week lab.

Background

This laboratory explores sequence labeling using a neural sequence tagger. By way of background, you might optionally try the following: 1. Work through this [introductory PyTorch tutorial](#). 2. Work through this [RNN tutorial](#), which has you train an RNN to classify surnames based on their language/ethnicity.

What are the top predictions for the name McNamee? How about Ocasio-Cortez? 3. We will be using the FLAIR LSTM-CRF system. Skim through [this paper](#) for details on how the system works.

Setting Up and Running the Tagger

This notebook describes how to run the FLAIR LSTM/CRF tagger on Colab. You will be using your Google drive to store your project code and data. In the following, most of the manipulation of files on your drive are done through Colab. However, it's fine to use the Google Drive interface if you prefer. You may also use a platform other than Colab if you prefer. If you use Colab, you will likely need to pay for a Colab Pro subscription to avoid timeouts (it costs far less than a textbook and can be canceled whenever you want).

This lab needs to run on a GPU. Click the 'runtime' menu above, select 'change runtime type' and verify you're using a GPU.

First, install the FLAIR and gensim packages. This can take as long as three minutes depending on your machinery.

```
!pip install flair --quiet
```

```
!pip install flair[word-embeddings]
```

You might see version errors, but they do not appear to affect the tagger's operation.

Import various modules.

```
import flair
from flair.embeddings import WordEmbeddings, FlairEmbeddings, StackedEmbeddi
from flair.models import SequenceTagger
from flair.trainers import ModelTrainer
from flair.data import Corpus
from flair.datasets import ColumnCorpus
from pathlib import Path
```

Mount your Google drive. You will be prompted to sign in to your Google account.

```
from google.colab import drive
gdrive_mount = '/content/drive'
drive.mount(gdrive_mount, force_remount=True)
```

Mounted at /content/drive

Create a directory to house your project, and move to it. The ! spawns a new process, while the % executes the command in the current process. Note that we need to put quotes around the directory name to run this in a Notebook because in their infinite wisdom Google has placed a space in the default gdrive path.

```
# Change this path if you'd like to work on your lab in a different director
labdir = '/content/drive/My Drive/Teaching/605.646/Fall 2025/lab05'
!mkdir -p "$labdir"
%cd "$labdir"
```

/content/drive/My Drive/Teaching/605.646/Fall 2025/lab05

Now, outside of Colab, obtain the file `conll03-englishversion.gz` from Canvas, unzip it, and place the resulting directory in your \$labdir. Notice that the data is broken into training, dev (called `valid.txt` here) and testing partitions.

```
datadir = f"{labdir}/conll03-englishversion"
!ls "$datadir"
```

metadata test.txt train.txt valid.txt

✓ Tagger Training

We're now ready to train a tagger. We'll start by naming our run.

```
runname = 'flair-ner-conll-eng3'
```

Next, we'll marshal the CoNLL 2003 English named entity recognition data.

```
# 1. Create the corpus
columns = {0: 'text', 1: 'pos', 2: 'chunk', 3: 'ner'}
corpus = ColumnCorpus(datadir, columns, train_file="train.txt", dev_file="va
print(corpus)
```

2025-10-26 20:18:29,451 Reading data from /content/drive/My Drive/Teaching/60
2025-10-26 20:18:29,453 Train: /content/drive/My Drive/Teaching/605.646/Fall
2025-10-26 20:18:29,453 Dev: /content/drive/My Drive/Teaching/605.646/Fall 20
2025-10-26 20:18:29,454 Test: /content/drive/My Drive/Teaching/605.646/Fall 2
Corpus: 14987 train + 3466 dev + 3684 test sentences

```
# 2. Select the label to predict
label_type = 'ner'
```

```
# 3. Create a label dictionary from the corpus
label_dict = corpus.make_label_dictionary(label_type=label_type, add_unk=False)
print(label_dict)
```

2025-10-26 20:18:46,475 Computing label dictionary. Progress:
1it [00:00, 781.94it/s]
14987it [00:00, 36071.33it/s]2025-10-26 20:18:46,907 Dictionary created for 1
Dictionary with 4 tags: LOC, PER, ORG, MISC

The tagger embeds its input using the combination of several models.

```
# 4. Initialize embedding stack with Flair and GloVe
embedding_types = [
    WordEmbeddings('glove'),
    FlairEmbeddings('news-forward'),
    FlairEmbeddings('news-backward'),
]
embeddings = StackedEmbeddings(embeddings=embedding_types)
```

2025-10-26 20:18:52,016 <https://flair.informatik.hu-berlin.de/resources/embed>
100%|██████████| 153M/153M [00:15<00:00, 10.1MB/s]2025-10-26 20:19:08,622 cop
2025-10-26 20:19:08,754 removing temp file /tmp/tmporzpwg7l1
2025-10-26 20:19:09,489 <https://flair.informatik.hu-berlin.de/resources/embed>
100%|██████████| 20.5M/20.5M [00:03<00:00, 6.71MB/s]2025-10-26 20:19:13,359 c

```

2025-10-26 20:19:13,372 removing temp file /tmp/tmpju9tgjky

2025-10-26 20:19:18,924 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 69.7M/69.7M [00:08<00:00, 9.05MB/s]2025-10-26 20:19:27,687 c

2025-10-26 20:19:27,775 removing temp file /tmp/tmpm30ncgb_
2025-10-26 20:19:30,163 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 69.7M/69.7M [00:08<00:00, 8.87MB/s]2025-10-26 20:19:39,053 c
2025-10-26 20:19:39,107 removing temp file /tmp/tmp1jr3obln

```

Now we'll set up a trainer for our model.

```

# 5. Initialize the sequence tagger
tagger = SequenceTagger(hidden_size=256,
                        embeddings=embeddings,
                        tag_dictionary=label_dict,
                        tag_type=label_type)

```

```

-PER, E-PER, I-PER, S-ORG, B-ORG, E-ORG, I-ORG, S-MISC, B-MISC, E-MISC, I-MISC

```

```

# 6. Initialize the trainer
trainer = ModelTrainer(tagger, corpus)

```

We're ready to train models using the `train` method. We can set a maximum number of epochs here. The model will fully train in fifty to sixty epochs. For most of your experiments, you will want to ensure that you run for the same number of epochs. Twenty epochs should be sufficient for most of your experiments.

You'll see at the start of the output a description of the tagger, indicating that it is a Bi-LSTM with a CRF on top and inputs supplied by a combination of embeddings.

```

# 7. Train the model
trainer.train(f'resources/taggers/{runname}',
             learning_rate=0.1,
             mini_batch_size=32,
             save_model_each_k_epochs=1,
             monitor_test=True,
             max_epochs=3)

```

Here are the files that we've built:

```
!pwd
!ls -l resources/taggers/$runname

/content/drive/MyDrive/Teaching/605.646/Fall 2025/lab05
total 2023320
-rw----- 1 root root 414160181 Oct 25 21:44 best-model.pt
-rw----- 1 root root    558449 Oct 25 21:44 dev.tsv
-rw----- 1 root root 414160198 Oct 25 21:45 final-model.pt
-rw----- 1 root root      878 Oct 25 21:36 loss.tsv
-rw----- 1 root root 414160232 Oct 25 21:39 model_epoch_1.pt
-rw----- 1 root root 414160232 Oct 25 21:41 model_epoch_2.pt
-rw----- 1 root root 414160232 Oct 25 21:44 model_epoch_3.pt
-rw----- 1 root root    506551 Oct 25 21:46 test.tsv
-rw----- 1 root root    10417 Oct 25 21:46 training.log
```

What are these files?

- `best-model.pt`: the best performing model encountered during training
- `dev.tsv`: space-separated token, ground truth label, and predicted label for the dev.txt file
- `final-model.pt`: The model as it stood when training ended
- `loss.tsv`: A truly tab-separated file with the following columns:

```
** EPOCH ** TIMESTAMP ** BAD_EPOCHS ** LEARNING_RATE ** TRAIN_LOSS ** DEV_LOSS
** DEV_PRECISION ** DEV_RECALL ** DEV_F1 ** DEV_ACCURACY
```

- `model_epoch_##.pt`: The model produced at the end of each epoch
- `test.tsv`: Same as dev.tsv but for test data
- `training.log`: Same as the output of the notebook cell

Note that most of the ".tsv" files are not really tab-separated; they're white space delimited.

- `weights.txt`: Model weights (must be specifically requested at training time; we won't use them).

✓ Tagging Text

We can tag any text we'd like using one of the models we've built. First, we load some packages.

```
from flair.data import Sentence
from flair.models import SequenceTagger
```

Now we'll load a saved model from disk.

```
model = SequenceTagger.load(f'{labdir}/resources/taggers/{runname}/final-mod
print(model)
```

```
2025-10-25 21:47:10,545 SequenceTagger predicts: Dictionary with 19 tags: 0,
SequenceTagger(
  (embeddings): StackedEmbeddings(
    (list_embedding_0): WordEmbeddings(
      '0-/root/.flair/embeddings/glove.gensim'
      (embedding): Embedding(400001, 100)
    )
    (list_embedding_1): FlairEmbeddings(
      (lm): LanguageModel(
        (drop): Dropout(p=0.05, inplace=False)
        (encoder): Embedding(300, 100)
        (rnn): LSTM(100, 2048)
      )
    )
    (list_embedding_2): FlairEmbeddings(
      (lm): LanguageModel(
        (drop): Dropout(p=0.05, inplace=False)
        (encoder): Embedding(300, 100)
        (rnn): LSTM(100, 2048)
      )
    )
  )
  (word_dropout): WordDropout(p=0.05)
  (locked_dropout): LockedDropout(p=0.5)
  (embedding2nn): Linear(in_features=4196, out_features=4196, bias=True)
  (rnn): LSTM(4196, 256, batch_first=True, bidirectional=True)
  (linear): Linear(in_features=512, out_features=19, bias=True)
  (loss_function): ViterbiLoss()
  (crf): CRF()
)
```

We're ready to tag some data. Let's create a sample sentence, then tag it.

```
from textwrap import wrap

# create example sentence
sentence = Sentence("The Washington Post reported that Washington DC team th
    " Washington Commanders visited George Washington's Mt. Vernon estat
# Use wrap to make sure output doesn't run off page
print('\n'.join(wrap(sentence.to_plain_string())), '\n')

# predict the tags
model.predict(sentence)
print('\n'.join(wrap(sentence.to_tagged_string())))
```

The Washington Post reported that Washington DC team the Washington
Commanders visited George Washington's Mt. Vernon estate.

Sentence[19]: "The Washington Post reported that Washington DC team

```
the Washington Commanders visited George Washington's Mt. Vernon
estate." → ["Washington Post"/ORG, "Washington DC"/ORG, "Washington
Commanders"/ORG, "George Washington"/LOC, "Mt. Vernon"/LOC]
```

✓ Assignment

A. (10 points) Train a named entity recognizer as described above on the `train.txt` partition of the CoNLL 2003 English NER data. This will be your 'base run.' The system will report dev scores at each iteration. Choose the number of epochs you will use for all of your runs. Within 20 iterations you should see F1 scores on the dev partition of 95% or better. You may choose a higher number if you want to see how good your system can get. You may choose a lower number if you are running out of resources, but be sure to run at least ten epochs. Report overall precision, recall, and F1 on the test set, as well as precision, recall, and F1 for each of the identified entity types. Create a plot of the (decreasing) loss values from the training process, where the x-axis is epoch and the y-axis is loss. Then, train the system three more times on the same test data. The purpose here is to see the the variation in scores caused by the randomness inherent in training a neural system. Report your minimum, mean (of the four values), and maximum overall F1 scores on the test data. Plot the F1 performance on both the dev and test data, where the x-axis is epoch and the y-axis is F1. You will find all of these data in `loss.tsv`.

B. (10 points) Examine how performance improves as more training data is used. The training file has about 220K lines. First, divide the data into 25K line chunks (preferably splitting between sentences). [Note: you might be able to use FLAIR's `Corpus.downsample` for this, although I have not tried that.] Then incrementally, train a system on the first chunk, the first two chunks, the first three chunks, etc. Run each on the same number of iterations as your base run. Plot the F1 learning curve as additional training data is made available, where the x-axis is the amount of training data and the y-axis is F1.

C. (10 points) Study how performance changes when we don't care about entity types. First, create copies of both the training and the test data and map all entity types in the copies onto a single entity type, ENT (i.e., map B-PER to B-ENT, I-PER to I-ENT, etc.). You will end up with just three tags: B-ENT, I-ENT, and O. Retrain the system on the new data, using the same number of iterations as you did in your first training. Compare the scores in overall precision, recall, and F1, when type information is included (using your base run) against this new run that maps all types to ENT. Describe why you think there is/is not a difference.

D. (10 points) Explore how the system works on examples of your own selection. First, create a new test file containing at least ten sentences. You should include at least ten named entities of each of the three types PER, ORG, and LOC. Your examples should include entities

you think will be easy to identify, as well as a variety of difficult cases. Run the tagger from the best saved model on your test examples.

Include a list of your sentences and the named entities the system found in your sentences in your pdf submission. Indicate whether the system failed on any of your easy examples, and describe how it fared on your difficult examples.

Submissions

As usual, your submission is to be a single pdf file. Please do not submit a huge file. There is no need to include outputs beyond what is requested. If your pdf is longer than twenty or thirty pages, you need to cut something out. Remember that Colab/Jupyter notebooks have a tendency to truncate outputs over a certain width or length. Please check your pdf submission to make sure that all the output you want to appear in it is actually there and has not run off the edge of the page.

Part A

```
runname = 'flair-ner-conll-eng3-partA'
```

```
trainer.train(f'resources/taggers/{runname}',  
              learning_rate=0.1,  
              mini_batch_size=32,  
              save_model_each_k_epochs=1,  
              monitor_test=True,  
              max_epochs=15)
```

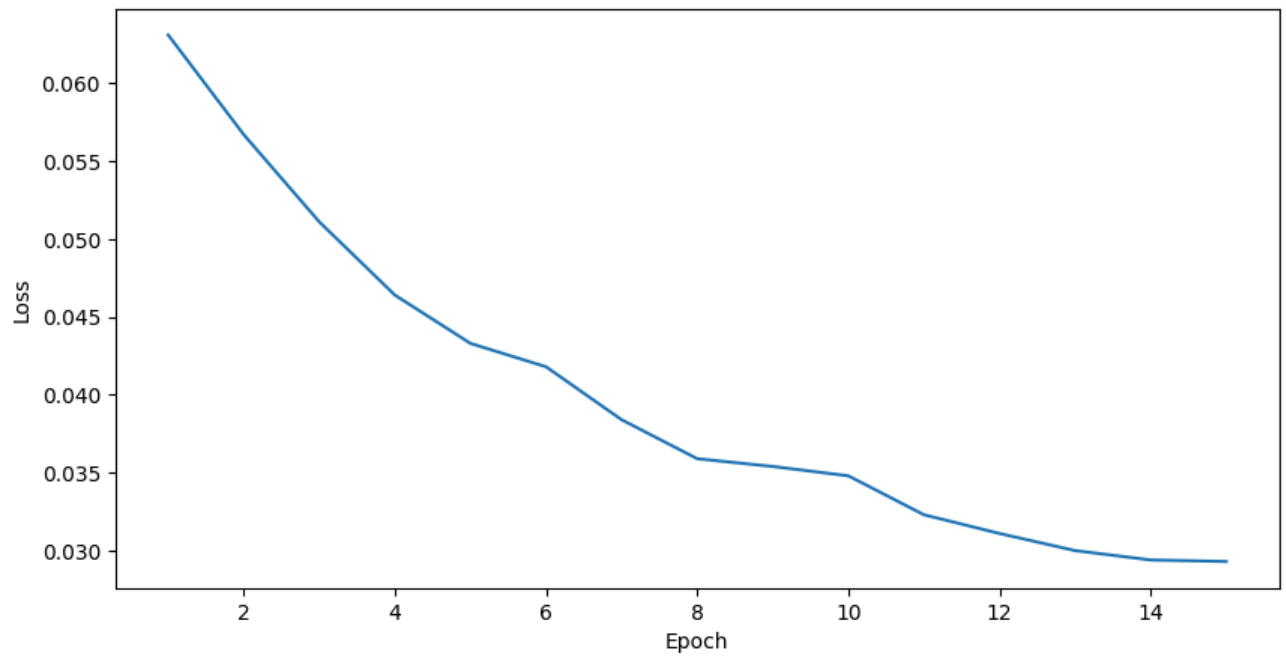
```
lossFile = f'resources/taggers/{runname}/loss.tsv'
```

```
import pandas as pd  
import matplotlib.pyplot as plt
```

```
df = pd.read_csv(lossFile, sep='\t')
```

```
plt.figure(figsize=(10, 5))  
plt.plot(df['EPOCH'], df['TRAIN_LOSS'])  
plt.xlabel('Epoch')  
plt.ylabel('Loss')
```

Text(0, 0.5, 'Loss')



```
runname = 'flair-ner-conll-eng3-partA-2'
```

```
trainer.train(f'resources/taggers/{runname}',  
              learning_rate=0.1,  
              mini_batch_size=32,  
              save_model_each_k_epochs=1,  
              monitor_test=True,  
              max_epochs=15)
```

```
runname = 'flair-ner-conll-eng3-partA-3'
```

```
trainer.train(f'resources/taggers/{runname}',  
              learning_rate=0.1,  
              mini_batch_size=32,  
              save_model_each_k_epochs=1,  
              monitor_test=True,  
              max_epochs=15)
```

```
runname = 'flair-ner-conll-eng3-partA-4'
```

```
trainer.train(f'resources/taggers/{runname}',  
              learning_rate=0.1,  
              mini_batch_size=32,
```

```
save_model_each_k_epochs=1,
monitor_test=True,
max_epochs=15)
```

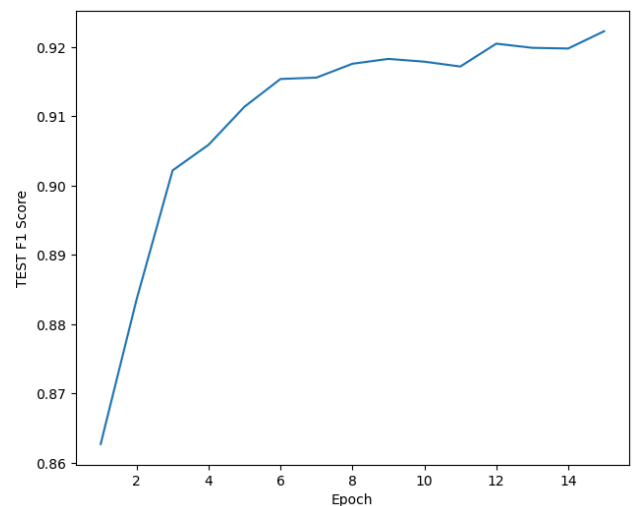
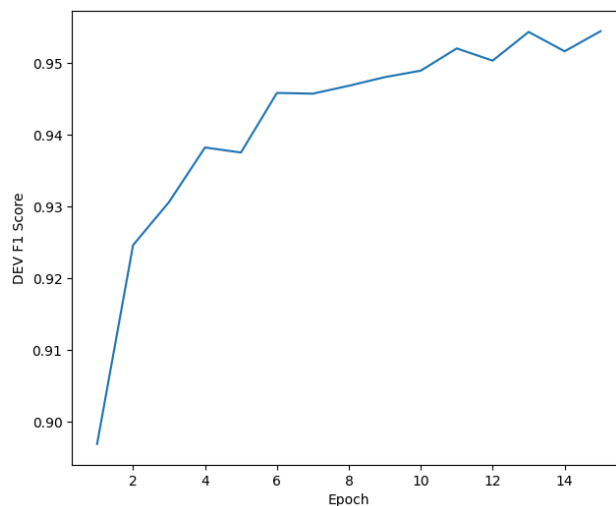
```
testF1s = []
```

```
loss_file = f'resources/taggers/flair-ner-conll-eng3-partA/loss.tsv'
df = pd.read_csv(loss_file, sep='\t')
testF1s.append(df['TEST_F1'].iloc[-1])
```

```
loss_file = f'resources/taggers/flair-ner-conll-eng3-partA-2/loss.tsv'
df = None
df = pd.read_csv(loss_file, sep='\t')
```

```
print("RUN 2")
fig, (p1, p2) = plt.subplots(1, 2, figsize=(16, 6))
p1.plot(df['EPOCH'], df['DEV_F1'])
p2.plot(df['EPOCH'], df['TEST_F1'])
p1.set_xlabel('Epoch')
p1.set_ylabel('DEV F1 Score')
p2.set_xlabel('Epoch')
p2.set_ylabel('TEST F1 Score')
testF1s.append(df['TEST_F1'].iloc[-1])
```

RUN 2



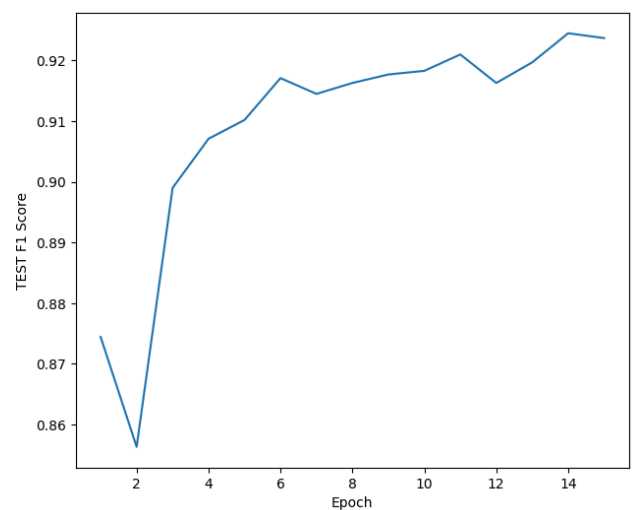
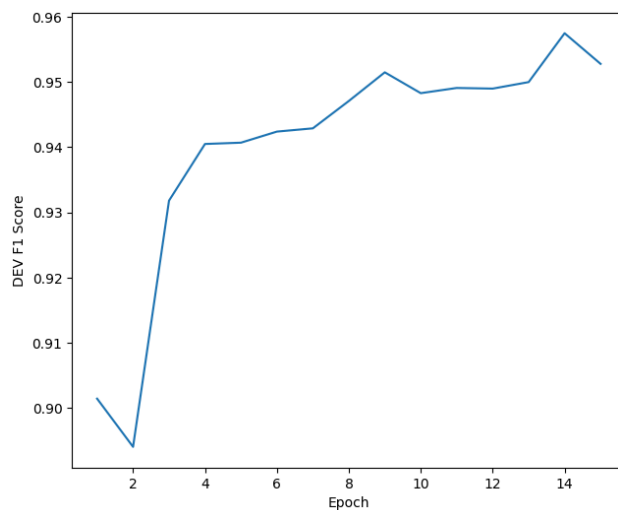
```
loss_file = f'resources/taggers/flair-ner-conll-eng3-partA-3/loss.tsv'
df = pd.read_csv(loss_file, sep='\t')
print("RUN 3")
fig, (p1, p2) = plt.subplots(1, 2, figsize=(16, 6))
p1.plot(df['EPOCH'], df['DEV_F1'])
p2.plot(df['EPOCH'], df['TEST_F1'])
```

```

p1.set_xlabel('Epoch')
p1.set_ylabel('DEV F1 Score')
p2.set_xlabel('Epoch')
p2.set_ylabel('TEST F1 Score')
testF1s.append(df['TEST_F1'].iloc[-1])

```

RUN 3

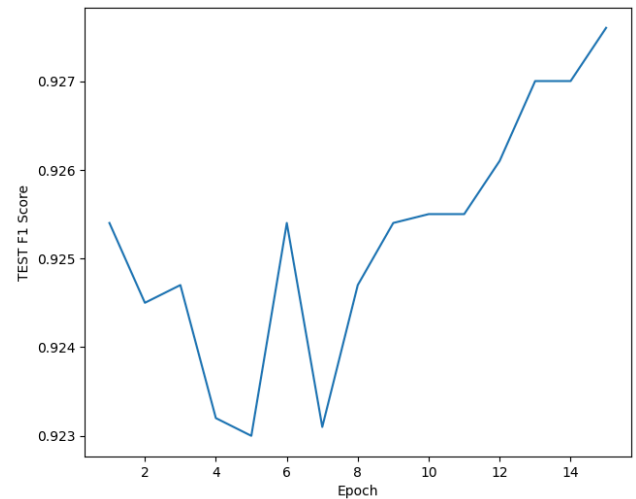
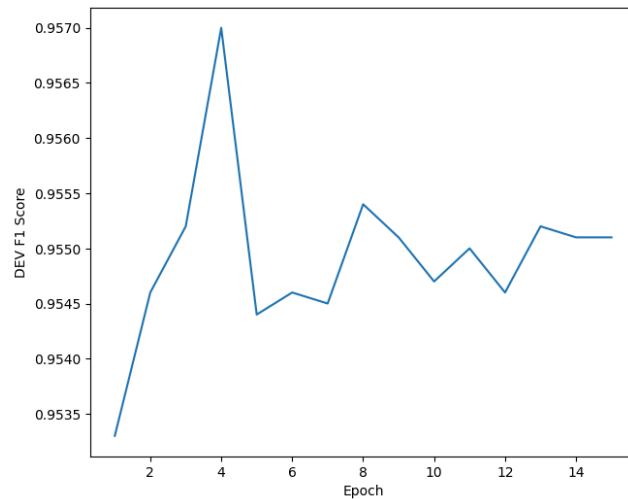


```

loss_file = f'resources/taggers/flair-ner-conll-eng3-partA-4/loss.tsv'
df = pd.read_csv(loss_file, sep='\t')
print("RUN 4")
fig, (p1, p2) = plt.subplots(1, 2, figsize=(16, 6))
p1.plot(df['EPOCH'], df['DEV_F1'])
p2.plot(df['EPOCH'], df['TEST_F1'])
p1.set_xlabel('Epoch')
p1.set_ylabel('DEV F1 Score')
p2.set_xlabel('Epoch')
p2.set_ylabel('TEST F1 Score')
testF1s.append(df['TEST_F1'].iloc[-1])

```

RUN 4



```
print("Test F1 Scores from All Runs:")
```

```
print(f"Min F1: {min(testF1s)}")
```

```
print(f"Mean F1: {sum(testF1s)/len(testF1s)}")
```

```
print(f"Max F1: {max(testF1s)}")
```

Test F1 Scores from All Runs:

Min F1: 0.9216

Mean F1: 0.9238

Max F1: 0.9276

Part B

```
percentages = [0.11,0.23,0.34,0.45,0.57,0.68,0.79,0.91,1.0]
results = []
```

```
i = 0
for p in percentages:
    runname = f'downsample-{i}'
    loss_file = f'resources/taggers/{runname}/loss.tsv'

    columns = {0: 'text', 1: 'pos', 2: 'chunk', 3: 'ner'}
    corpus = ColumnCorpus(datadir, columns,
                          train_file="train.txt",
                          dev_file="valid.txt",
                          test_file="test.txt")

    if p < 1.0:
        corpus = corpus.downsample(percentage=p, downsample_train=True,
                                   downsample_dev=False, downsample_test=False)
```

```

label_type = 'ner'
label_dict = corpus.make_label_dictionary(label_type=label_type, add_unk=F

embedding_types = [
    WordEmbeddings('glove'),
    FlairEmbeddings('news-forward'),
    FlairEmbeddings('news-backward'),
]
embeddings = StackedEmbeddings(embeddings=embedding_types)
tagger = SequenceTagger(hidden_size=256,
                        embeddings=embeddings,
                        tag_dictionary=label_dict,
                        tag_type=label_type)

trainer = ModelTrainer(tagger, corpus)

trainer.train(f'resources/taggers/{runname}',
              learning_rate=0.1,
              mini_batch_size=32,
              save_model_each_k_epochs=1,
              monitor_test=True,
              max_epochs=15)
loss_file = f'resources/taggers/{runname}/loss.tsv'
df = pd.read_csv(loss_file, sep='\t')
final = df.iloc[-1]

results.append({
    'percentage': p,
    'numOfTrain': len(corpus.train),
    'dev_f1': final['DEV_F1'],
    'test_f1': final['TEST_F1']
})

i+=1

```

results

```

[{'percentage': 0.11,
  'numOfTrain': 1649,
  'dev_f1': np.float64(0.8942),
  'test_f1': np.float64(0.8638)},
 {'percentage': 0.23,
  'numOfTrain': 3447,
  'dev_f1': np.float64(0.9237),
  'test_f1': np.float64(0.8982)},
 {'percentage': 0.34,
  'numOfTrain': 5096,
  'dev_f1': np.float64(0.9381),
  'test_f1': np.float64(0.9095)},

```

```
{'percentage': 0.45,
 'numOfTrain': 6744,
 'dev_f1': np.float64(0.9393),
 'test_f1': np.float64(0.9103)},
{'percentage': 0.57,
 'numOfTrain': 8543,
 'dev_f1': np.float64(0.9488),
 'test_f1': np.float64(0.915)}]}
```

Start coding or [generate](#) with AI.

I only did it up to a few chunks because I was running out of google drive storage and it was taking a lot of compute units.

```
df = pd.DataFrame(results)
```

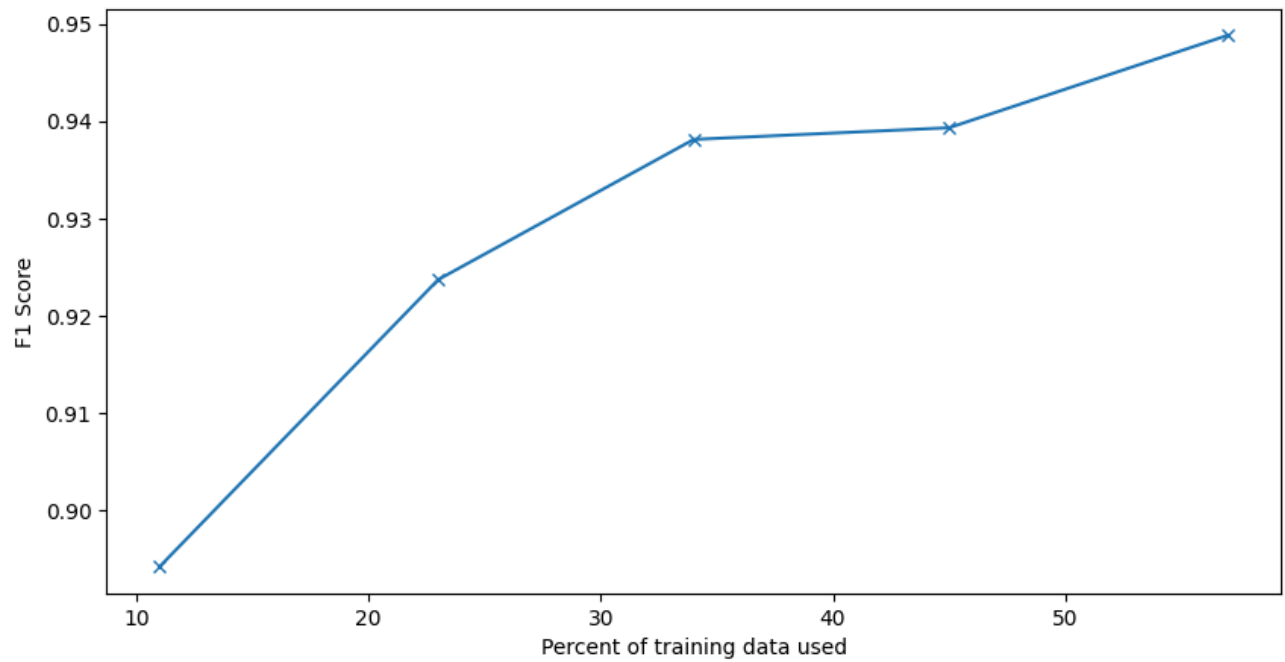
df

	percentage	numOfTrain	dev_f1	test_f1	
0	0.11	1649	0.8942	0.8638	
1	0.23	3447	0.9237	0.8982	
2	0.34	5096	0.9381	0.9095	
3	0.45	6744	0.9393	0.9103	
4	0.57	8543	0.9488	0.9150	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
fig, p = plt.subplots(figsize=(10, 5))
p.plot(df['percentage']*100, df['dev_f1'],marker='x')
p.set_xlabel('Percent of training data used')
p.set_ylabel('F1 Score')
```

Text(0, 0.5, 'F1 Score')



Start coding or [generate](#) with AI.

Part C

```
runname = 'flair-ner-conll-eng3-newEntity'
```

```
%cd "$labdir"
```

```
/content/drive/My Drive/Teaching/605.646/Fall 2025/lab05
```

```
newdatadir = f"{labdir}/conll03-englishversion-new"
!ls "$newdatadir"
```

```
metadata test.txt train.txt valid.txt
```

```
# 1. Create the corpus
```

```
columns = {0: 'text', 1: 'pos', 2: 'chunk', 3: 'ner'}
```

```
newcorpus = ColumnCorpus(newdatadir, columns, train_file="train.txt", dev_file="valid.txt")
print(newcorpus)
```

```
2025-10-27 00:09:52,570 Reading data from /content/drive/My Drive/Teaching/605.646/Fall 2025
2025-10-27 00:09:52,570 Train: /content/drive/My Drive/Teaching/605.646/Fall 2025/train.txt
2025-10-27 00:09:52,571 Dev: /content/drive/My Drive/Teaching/605.646/Fall 2025/valid.txt
```

```
2025-10-27 00:09:52,571 Test: /content/drive/My Drive/Teaching/605.646/Fall 2
Corpus: 14987 train + 3466 dev + 3684 test sentences
```

```
label_type = 'ner'
```

```
label_dict = newcorpus.make_label_dictionary(label_type=label_type, add_unk=
print(label_dict)
```

```
2025-10-27 00:10:07,329 Computing label dictionary. Progress:
1it [00:00, 1346.05it/s]
14987it [00:00, 39341.39it/s]2025-10-27 00:10:07,721 Dictionary created for l
Dictionary with 1 tags: ENT
```

```
# 4. Initialize embedding stack with Flair and GloVe
embedding_types = [
    WordEmbeddings('glove'),
    FlairEmbeddings('news-forward'),
    FlairEmbeddings('news-backward'),
]
embeddings = StackedEmbeddings(embeddings=embedding_types)
```

```
2025-10-27 00:10:10,251 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 153M/153M [00:07<00:00, 20.7MB/s]2025-10-27 00:10:18,368 cop
```

```
2025-10-27 00:10:18,442 removing temp file /tmp/tmp2qqm2q6g
2025-10-27 00:10:18,835 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 20.5M/20.5M [00:01<00:00, 13.3MB/s]2025-10-27 00:10:20,828 c
2025-10-27 00:10:20,838 removing temp file /tmp/tmpfl5ocmxo
```

```
2025-10-27 00:10:24,430 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 69.7M/69.7M [00:03<00:00, 19.1MB/s]2025-10-27 00:10:28,613 c
```

```
2025-10-27 00:10:28,647 removing temp file /tmp/tmp70pi1rjf
2025-10-27 00:10:29,818 https://flair.informatik.hu-berlin.de/resources/embed
100%|██████████| 69.7M/69.7M [00:03<00:00, 18.6MB/s]2025-10-27 00:10:34,129 c
2025-10-27 00:10:34,163 removing temp file /tmp/tmpgkvk1c5me
```

```
# 5. Initialize the sequence tagger
tagger = SequenceTagger(hidden_size=256,
                        embeddings=embeddings,
                        tag_dictionary=label_dict,
                        tag_type=label_type)
```

```
2025-10-27 00:11:01,189 SequenceTagger predicts: Dictionary with 5 tags: 0, S
```

```
trainer = ModelTrainer(tagger, newcorpus)
```

```
# 7. Train the model
trainer.train(f'resources/taggers/{runname}',
              learning_rate=0.1,
              mini_batch_size=32,
              save_model_each_k_epochs=1,
              monitor_test=True,
              max_epochs=15)
```

For the base run, the precision was 0.9178, the recall was 0.9240 and the f1-score was 0.9209. For this new run mapped entities, Precision is 0.9491, recall is 0.9635, and f1-score is 0.9562. I believe the performance was overall better in this new run because it just has to predict if it is in one category or not. In the base run it had to predict if it was an identity and which type of identity it was.